

Research Article

A Novel Approach for Enhancing Plant Leaf Classification With the Binary Cuckoo Search Algorithm

Mohammad Subhi Al-Batah ¹, Mohammad Ryiad Al-Eiade ¹ and Yazan Alnsour²

¹Department of Computer Science, Faculty of Information Technology, Jadara University, Irbid, Jordan

²Department of Management Information Systems, College of Business Administration, Prince Mohammad bin Fahad University, Khobar, Saudi Arabia

Correspondence should be addressed to Mohammad Subhi Al-Batah; albatah@jadara.edu.jo

Received 14 May 2024; Accepted 29 January 2025

Academic Editor: Abidhan Bardhan

Copyright © 2025 Mohammad Subhi Al-Batah et al. Applied Computational Intelligence and Soft Computing published by John Wiley & Sons Ltd. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Computer vision plays a crucial role in current day tasks due to its wide range of merits and applications in many fields like disease diagnosis, medical decisions, military, security, scientific applications, and business. Identifying plant species upon their leaf images is a very challenging and complex task. In this study, we introduced a model for classifying a variety set of plant leaves using different techniques such as factorization machine (FM), dimensionality reduction (DR), and ensemble learning (EL). In our study, FM is used as a classification algorithm, whereas DR is included in two stages, namely, feature extraction (FE) and feature selection (FS). FE has been used to extract the features from images in tabular form; on the other hand, FS is used to reduce the size of the data by getting rid of noisy (redundant and irrelevant) features. In addition, we utilized two methods for FS, filter-based and wrapper-based approaches. The used filter was the minimum redundancy maximum relevance, and the used wrapper was the improved binary cuckoo search algorithm that is based on Lévy flight and abandon nest functions. Regarding EL, it was used to declare different versions of FM by passing different subsets of features that are deduced from the original ones to improve its performance. We used the Swedish and Flavia leaf datasets for training and testing phases. The proposed model achieved a high performance in terms of accuracy that produced 95.67% on Swedish dataset and 99.6% on Flavia dataset. According to the results and in comparison to other methods, we proved that our proposed model ensures a favorable plant leaf classification approach. Finally, the proposed wrapper FS approach showed very good results without setting up the number of features to be selected by the user. The entire implementation of this work can be found at <https://github.com/Mohammed-Ryiad-Eiadeh/Binary-Cuckoo-Search-For-Plant-Leaf-Prediction>.

Keywords: binary cuckoo search algorithm; binary transfer function; dimensionality reduction; ensemble learning; factorization machine; plant classification; Tribuo machine learning library

1. Introduction

Plants are considered a major and an indispensable part of our ecosystem. They play an important role in preserving nature and keeping the balance in our atmosphere for all kinds of creatures. Plants are essential resource for food, mediation, and fuel [1]. Plants are pivotal to the atmosphere as they play an important role in consuming carbon dioxide from the air to produce oxygen which is crucial for our survival. With recent advancements, mankind had been

taking advantage of such an important resource to use in different industries such as furniture, cosmetic, agriculture, pharmacology and many others. For example, herbs have been utilized as a type of medicine by people in many nations since the dawn of civilization. Their benefits have been recognized by humans who experiment with such resources based on personal or olfactory sense [2].

Generally, identifying objects based on images is a very promising endeavor and attracts the attention of scholars in areas like image processing, pattern recognition, and

computer vision [3–9]. The leaf is an important part of determining the type of plant as it holds rich information that provides important description of the plant's properties [5]. Leveraging artificial intelligence (AI) in plant classifications requires a certain level of skills and professional knowledge in the characteristics of a variety set of plants. Nonetheless, expert scientists are facing several challenges in such a task which brings the need to developing and adopting automatic plant classifying models in order to enhance the performance and the reliability of plant classification [6]. Reviewing the recent body of work, researchers have employed algorithms such as K-Nearest Neighbor (KNN), feedforward backpropagation multilayered perceptron, support vector machine (SVM), neuro fuzzy controller, and others, yet it is noticeable that the improper feature extraction (FE) and selection provided irrelevant and redundant features that limited different models' performance [7]. In addition, some of the deep learning (DL) algorithms have been used in image classification and achieved a high accuracy in such tasks, for example, ResNet 50 [8] and MobileNet [9].

In this study, we first applied FE using the Orange tool. Following that we conducted feature selection (FS) based on minimum redundancy maximum relevance (mRMR) filter and binary cuckoo optimization algorithm. Then we used the factorization machine (FM) classifier with ensemble learning (EL) based on bagging method for plant leaf classification. Finally, we evaluated our model with other approaches to experimentally test the performance of our model and show its capability to tackle image classification over different benchmark leaf datasets.

2. Related Work

For more and clear understanding of the related body of work and the existing literature that had been conducted for plant identification using the leaf characteristics, in this section, we provide an overview and a summary of a number of proposed studies that had been initiated to develop plant leaf classification systems based on DL and machine learning (ML) methods.

Pankaja and Suma proposed a hybridization model that combined whale optimization algorithm (WOA) [10] with random forest (RF) classifier, and the structure of the developed model included three stages. First stage converted images to grayscale for decreasing the difference in color data, converting foreground and background images to binary based on certain threshold, and standardizing the size of all images using morphological operations like erosion and dilation. Second stage contained FE of three different types of features: texture, shape, and color features, where a modified local binary pattern (LBP) was used for texture FE. In addition, the scholars incorporated eccentricity, rectangularity, circularity, invariant moments, aspect ratio, and perimeter to shape feature extraction. Also, metrics like mean, variance, skewness, and kurtosis were utilized for color FE. For the last stage, the researchers used WOA for FS to drop noisy features and RF learner to evaluate each subset of features to select the better one [7].

Another recent work by Kaur et al. proposed a leaf identifying system based on supervised ML learners and their proposed model included three stages. First stage contained data and image acquisition method to collect and construct their own data which included 10 categories like mint and tulsi. Second stage contained FE and FS, where leaf's height, width, area, perimeter, aspect ratio, rectangularity, circularity, red mean, blue mean, green mean, red, blue, and green standard deviation were extracted from images in a tabular form, and contrast, correlation, inverse different moments, and entropy were utilized for dropping the noisy features as cleaning process. Last stage contained training and evaluating several ML trainers such as SVM, KNN, RF, and more [11]. On the other hand, Ibrahim, Sabri, and Mangshor proposed a system for identifying herbal plant based on the leaf texture features and SVM. Their proposed model included three stages. First stage contained several preprocessing: image resizing and converting the image into grayscale style. Second stage contained FE of three types of texture features which are histogram of oriented gradients (HOG), LBP, and speeded up robust features (SURF). Last stage contained the process of training and evaluating SVM classifier. Moreover, they constructed a new dataset of 10 types of leaves [12].

Contrary to the above, Xu proposed a scale-generation approach based on shape descriptor with KNN and SVM classifiers, and the model included only two stages. First stage contained the extraction of leaf contour which was extracted by converting original images to grayscale, then converting them to binary form, and finally extracting the shape of the leaf. Subsequently, the scale-generation method employed trisection on the leaf contour to determine all paired neighbor points under various scales. Then those points were used to extract angle information to form multiscale angle representation to make it compact and measure similarity easily. Second stage contained training and evaluating KNN (where $K = 1$) and SVM for classification [5].

Asim et al. proposed a leaf classification system using multifeature analysis with several ML classifiers, and the model included three stages. First stage contained data acquisition by capturing 12 leaf type features of guava by a digital camera. Second stage contained FE using different methods like histograms, binary features, textures, rotational, spectral, and translational features. A correlation-based FS was operated on 47 extracted features and decreased to 18 features. Last stage contained the process of training and testing ML classifiers, instant base identifier, RF, and meta bagging [13]. A recent work by Yang proposed a plant leaf recognition system based on texture and shape features, and the model also included two stages. First stage contained a new integrated method between shape and texture properties for FE, but this method was erected manually which is time consuming and complex and this is the main drawback of this model. Multiscale triangle descriptor was used for extracting shape features, and LBP histogram Fourier was used for extracting texture features from leaf images. The second stage contained training and testing for classification purposes [14].

Another thought-provoking work by Zhu et al. proposed a plant leaf identification system based on an improved deep neural network, and their model comprised of three stages. First stage contained the process of improving deep neural network performance by using the batch normalization from (Inception-V2) [15] instead of the convolutional neural layers in faster region convolutional network. Second stage involved resizing the original images and then applying a segmentation method for leaf mask extraction; subsequently, those masks were loaded to the improved network iteratively. Last stage continued the classification process which was carried out by bounding box regressor and the softmax layer in order to provide the recognition result with the mask below the outputted label [16]. Another paper by Rojas-Hernandez et al. proposed a plant identification system based on new geometric features deduced from images and supervised ML algorithms, and the model included three stages. First stage contained image scaling, normalization, and segmentation. Image scaling was used to select certain independence of the image properties like brightness and contrast by some calculations like larger and smaller pixels from each image. A spatial thresholding-based probability distribution and correlation between pixels were used for leaf segmentation. Second stage contained FE where pixels of the leaf region, distance around the boundary of leaf region, smallest convex polygon that encloses the leaf region, area of convex hull, and distance around the boundary of convex hull were constructed. Last stage contained the training and evaluation of decision tree C4.5, RF, multilayer perceptron, logistic regression, KNN, Naïve Bayes, and SVM classifiers [17].

Shao proposed a novel manifold learning approach called supervised global locality preserving projection, and the model included two stages. First stage contained the usage of local and class information of the training instances for creating weighted inter- and intraclass scatter matrix. Subsequently, it separately enhanced the compactness of intraclass points and interclass manifold by designing a multiobjective optimal solution. Last stage contained the construction of global weighted interclass scatter matrix to expand the distance between different classes of the data and then expose the intrinsic manifold structure for classification [18].

Cao, Wang, and Brown proposed a novel shape descriptor named R-angle for measuring the similarity between image leaves, and the model included two stages. First stage contained the process of measuring the angle between the crossings of shape contour of the leaf with a circle radius R which is centered at points sampled around the contour in order to describe the curve of the contour. Subsequently, the model introduced the notation of scale transformer which in turn was utilized to produce a coarse-to-fine description of the local curve of the contour. Furthermore, a local scale arrangement was achieved by computing the distance between each contour point and the scale for the given point. Last stage contained two matching approaches in order to measure the similarities and dissimilarities between the leaves [19].

Wang et al. proposed a novel representation of leaf shape named bag of contour fragments which is elated from the classic bag of word approach, and this model included four

stages. First stage contained the usage of shape context descriptor to demonstrate the contour fragments which was fragmented from the shape. Second stage contained the encryption process of the contour fragments into a shape code. Third stage contained the usage of pooling shape codes to construct the compact shape representation. Last stage contained the training and evaluating process of SVM for classification purposes [20]. Alajlan et al. proposed triangle area representation and dynamic space wrapping for leaf identification system, and the model included two stages. The first stage involved constructing a representation based on triangular areas formed by boundary points, which were used to measure the convexity or concavity (or the lengths of triangle sides) at various scales. This approach effectively captures local shape properties and is invariant to translation, rotation, and scaling, while remaining robust against noise and moderate occlusion. Last stage contained the utilization of dynamic space wrapping to search about the optimal correspondence between the points of two shapes to compute the distance between them [21].

In addition, the previous work, while reviewing the literature we came across the particle swarm optimization algorithm that was inspired by the behavior folks of birds when they seek food. Similar to the birds in folks each particle is posed a solution in the search space for a certain problem [22, 23]. Additionally, the Black Hole Optimization Algorithm is mentioned, which is inspired by the behavior of stars moving toward a black hole due to its gravitational pull. In this analogy, the best solution in the search space is considered analogous to a black hole, attracting other candidate solutions (stars) toward it [24]. The humpback whale optimization algorithm was inspired by the whales' hunting strategy which is divided into three stages, searching, encircling, and spiral bubble net attacking [10]. Also, the genetic algorithm (GA) is one of the popular and widely used optimization algorithms that is deduced from the principles of natural selection and genetic variation. It gained huge popularity because of its simple structure and its capabilities to provide a wide set of solutions. GA is divided into four main stages, namely, the chromosome (solutions) initialization, selection, mutation, and crossover operator [25]. Another example is the gravitational search algorithm that was inspired by the Einstein relativism theory. This theory states that interrail and free-falling systems are equivalent. Einstein principle of equivalence states that acceleration of free-falling laboratory cancels the affection of gravity [26]. Many evolutionary computation algorithms are used for FS tasks and showed that they could be good alternatives in comparison to the state-of-the-art methods.

The increasing demand that was initiated from industrialization has driven the development of advanced optimization techniques, resulting in the rise of meta-heuristic algorithms, with over 300 new methods emerging in the past decade. For example, bio-inspired optimization strategies, which mimic natural and biological processes, have gained prominence due to their effectiveness in solving complex engineering problems, offering fast convergence and superior fitness outcomes. This study examines various bio-inspired algorithms, focusing on their operational

mechanisms, exploration–exploitation balance, and performance against standard benchmark functions, such as those provided by the Congress on Evolutionary Computation benchmark suite. Comparative evaluations, including convergence curves, trajectory plots, and box plots, reveal that these approaches surpass traditional methods, demonstrating their suitability for handling intricate challenges. Furthermore, we discuss the scalability of these algorithms for handling large-scale problems and their performance when changing the hyperparameters to check how sensitive their behavior is to the parameter variability. Evolutionary and swarm intelligence–based algorithms, in particular, provide adaptive and efficient solutions with enhanced convergence rates. The findings emphasize the potential of bio-inspired optimizers and point to future research opportunities for further refinement and innovation in nature-inspired optimization techniques [27].

Seeking patterns, the artificial neural networks (ANNs) replicate brain-like data processing using interconnected neurons (processing units) that learn to solve problems without providing direct instructions similar to traditional programming. ANNs are commonly applied in classification, regression, and clustering tasks. Previous work [28] leveraged ANNs' functionality and discussed their advantages and applications, such as image processing and MNIST. Also, research such as [29] examined the application of hyperspectral remote sensing in agriculture, incorporating the fly optimization algorithm (FOA) to enhance image acquisition. With an 83% success rate, FOA enhances the effectiveness of hyperspectral imaging, surpassing existing methods. The approach aids agriculturalists in efficiently managing and storing geographic metadata.

3. Methodology

In this section, we explained our proposed approach and how we can tackle the problem by using a combination of Orange tool, FE (e.g., mRMR), EL (e.g., bagging), and FM classifier. In addition, we add a lot of figures and samples to clarify with more details the proposed approach. In Figure 1, we demonstrate the general waterfall model of our work.

In the following sections, we will clarify the stages appearing in Figure 1 to explain our proposed model. Furthermore, we will demonstrate and explain the key points of the model in detail and illustrate some figures and equations. The remainder of the paper is organized as follows: Section 4 presents the experimental results, Section 5 provides the conclusion, and the final part includes the references.

3.1. Image FE Using Orange Tool. Orange data mining is a powerful platform for performing a variety set of ML tasks. It has the capabilities to perform data analysis with modern user interface (UI) and data visualization. It provides statistical distributions of data and displays different plot types (e.g., scatter plots). In addition, the Orange tool contains various algorithms to perform tasks such as classification and clustering. Furthermore, many feature engineering algorithms are available in Orange tool like feature ranking. It

is flexible especially in cases when extending the functionality and adding new toolbox like image analytics which contains five UI nodes that are (1) Import Images that is used to import images from a specific directory, (2) Image Viewer that is used to view the input images, (3) Image Embedded that is used to extract numerical features from image into two dimensions by utilizing different deep neural networks (e.g., Inception V3 [30], VGG-16 [31], and OpenFace [32]), (4) Image Grid that is used to visualize image in a similarity grid, and (5) Save Images which is used to save images in a specific directory. To get more information about Orange ML tool, go to <https://orangedatamining.com/>.

For illustration, assume we have two folders, each containing 50 images. Figure 2 depicts how to use this tool to perform FE. Each folder is imported separately, and FE is applied to extract the features. The extracted features are then saved as a CSV file, and the labels are manually assigned for each of the 50 instances (tuples). In real-life datasets, instances are not arranged sequentially, for example, the first 50 images with label “label 1” and the next 50 images with label “label 2.” To address that shuffling, the data were used to mimic real-life applications.

3.2. FS in ML. The rapid improvement of digitization in different fields has introduced clusters of high-dimensional datasets that are deduced from diverse sources like sensor network, financial systems, health care, social media, bio-informatics, and others [33]. Many of the newly developed datasets contain irrelevant, redundant, and noisy features that have negative impact such as slowing down and decreasing the performance of ML algorithms and consuming additional resources (e.g., RAM and CPU) [34]. Such datasets make the process of data analysis very complex and hard to produce something meaningful from it. The procedure of getting rid of noisy features is called “minimal optimal” which is defined as a convenient way to select possible subset with features that can improve the performance of classification algorithms [35].

Dimensionality reduction (DR) techniques are very efficient processes that can be performed on data to reduce their complexity. Those techniques are divided into three types: feature construction (FC), FE or feature projection, and FS (see Figure 3). FC involves enhancing the representational power of the original feature set by constructing additional features. This process reveals relationships among features and expands the feature space to capture more informative patterns. In other words, FC attempts to transform the original representation space of features (fields) to new ones in order to improve the performance of ML algorithms [36, 37]. FE aims to extract the topmost significant features from the original data by some transformation functions, and the size of the produced set of features is lower than the size of the original data that could be images or texts. On other hand, FS attempts to select the optimal subset of features from the original data to increase the classification accuracy [38]. Also, FS improves the classifier performance by reducing the volume of data without a significant dip in performance. It is a useful technique for models that tend to work with massive and

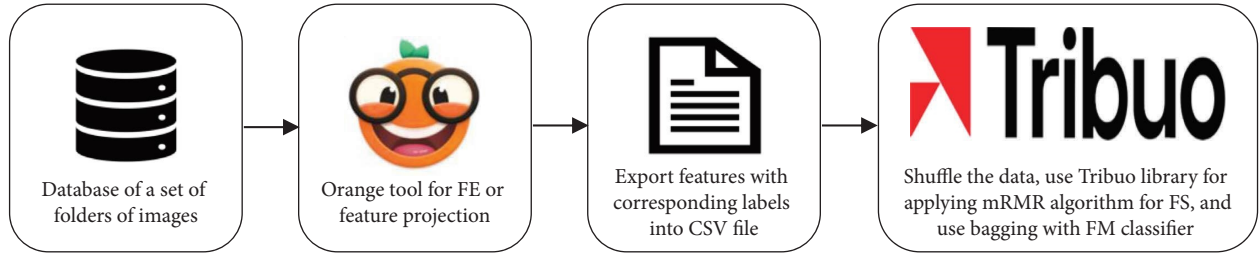


FIGURE 1: The general stages of the proposed model.

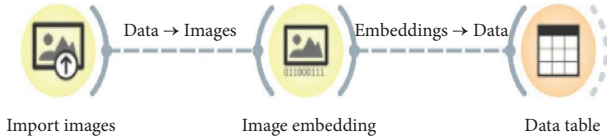


FIGURE 2: The diagram of using Orange with Inception V3 for extracting features from images.

complex problems. FS has been utilized in a variety of research fields like image processing (e.g., image recognition), text mining (e.g., text clustering), industrial applications (e.g., fault diagnosis), bioinformatics (e.g., biomarker discovery) [39], biomedical engineering, electrical and electronic engineering, computer engineering, and detection systems (e.g., phishing URL detection).

FS algorithms fall into three major categories, filter, wrapper, and embedded algorithms [40]. Filter algorithms aim to assess subsets of features from the original dataset without any intervention of the certain classifier, and these approaches use statistical functions to rank each feature as an indicator of its importance [41]. Wrapper algorithms aim to select features with a learner intervention, and the

rationale behind these algorithms is to generate subsets of features, evaluate each one using a certain learner, and then finally select the optimal subset [42]. Embedded algorithms are somewhat similar to wrapper approaches as both of them tend to optimize an objective function of a certain classifier based on an intrinsic model which constructs a scale value that is employed during the learning process [43].

3.3. Removing Noisy Features Using mRMR Algorithm. In this study, we used a heuristic algorithm for FS that is called mRMR filter [44, 45]. To clarify this approach, we produced a brief introduction with some symbols that are more convenient to present. That being said, given a dataset D of N -instances, M -features $X = \{x_i, i = 1, 2, \dots, M\}$, and the target class label l , the goal is to find a subspace R^m that optimally depicts and characterizes l deduced from M -dimensional space R^M . Finding the optimal R^m requires the use of an efficient search algorithm, such as the cuckoo search (CS) algorithm (Algorithm 1) proposed by Yang and Deb [46], since the total number of subsets in R^m is $2^m - 1$, and the number of subsets which have features lower than m is

$$\sum_{i=1}^m c(i, m) = \sum_{i=1}^m \binom{m}{i} = \sum_{i=1}^m \frac{M!}{i!(M-i)!}, \quad c \text{ is binomial coefficients.} \quad (1)$$

According to equation (1), utilizing an exact search is very costly. Therefore, researchers tend to use sequential search-based approximation functions such as sequential forward floating search [47]. To optimally characterize l , it is crucial to determine the dependency between x_i and l which is called maximal dependency (MD) [48]. MD can be carried out by maximal relevance (MR) for FS which tends to select features in which they are marked as the highest relevance to the target label l . MR can be achieved by measuring the correlation [49], or mutual information (MI) [50]. In this work, we discussed ML-based FS. MI of two random variables u and z is given by their probabilistic density functions $p(u)$, $p(z)$, and $p(u, z)$ which can be defined as follows:

$$MI(u, z) = \iint p(u, z) \log \frac{p(u, z)}{p(u)p(z)} du dz. \quad (2)$$

In MR, MI is required for each variable x_i individually with label l (denoted by $MI[x_i, l]$). Moreover, constructing a set of individually promising features does not necessarily mean that it is a good set of features which is desired to improve the classifiers' performance. In order to select a subset of features with no redundancy, we can use many approaches like prefiltering method to group variables [51] which is used to remove redundant variables. Due to the term of MI-based FS, the task is to find a subset S of m features in which they jointly have the largest dependency with respect to target label l , and MD-based FS can be defined as follows:

$$\max D(S, l), \quad D = MI(\{x_i, i = 1, \dots, m\}, l). \quad (3)$$

Initially, S starts with one feature, where S is subjected to size $|S| \leq m$. Subsequently, a simple incremental search function was used to add one feature at one time such that

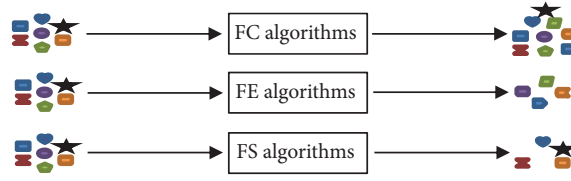


FIGURE 3: The difference between FC, FE, and FS.

for a given subset S of size $m - 1$, the feature x_m is added only and only if x_m increases $MI(S, l)$, which can be defined as follows:

$$\begin{aligned}
 MI(S_m, l) &= \iint p(S_m, l) \log \frac{p(S_m, l)}{p(S_m)p(l)} dS_m dl \\
 &= \iint p(S_{m-1}, x_m, l) \log \frac{p(S_{m-1}, x_m, l)}{p(S_{m-1}, x_m)p(l)} dS_{m-1} dx_m dl \\
 &= \int \cdots \int p(x_1, \dots, x_m, l) \log \frac{p(x_1, \dots, x_m, l)}{p(x_1, \dots, x_m)p(l)} dx_1, \dots, dx_m dl.
 \end{aligned} \tag{4}$$

MD has some complications when working with high-dimensional data such as the difficulties of getting an accurate estimation for multivariate density if the number of samples is insufficient, if there is slow computation speed, and if there is hardness of MD implementation. mRMR tends to select features with a high relationship with the target class and a low relationship among themselves [52]. The first component, MR, represents the average of all MI values between each individual feature x_i in subset S and the target class l . MR is redefined as follows:

$$\max D(S, l), \quad D = \frac{1}{|S|} \sum_{x_i \in S} MI(x_i, l). \tag{5}$$

In real-life tasks and datasets, selecting features according to MR may provide a subset of redundant features with a high dependency among them. Therefore, minimal redundancy R is used to select features with no dependency and redundancy. R is defined as follows:

$$\min R(S, l), \quad R = \frac{1}{|S|^2} \sum_{x_i, x_j \in S} MI(x_i, x_j). \tag{6}$$

mRMR filter combines R and MR and keeps looking for adding a new feature at one time in which this adding would increase the ingredient of subtracting R from MR. In other words, mRMR aims to maximize the relevance between the target class and the features in a given subset and maximize the minimization of the correlation and redundancy of the features in the given subset. Moreover, adding a new feature to S can be achieved using some incremental search methods. Assume we have S_{m-1} , and the task is to select the m th feature from $X - S_{m-1}$. Then the searching method should optimize the following rule:

$$\max_{x_j \in X - S_{m-1}} \left[MI(x_j, l) - \frac{1}{m-1} \sum_{x_i \in S_{m-1}} MI(x_j, x_i) \right]. \tag{7}$$

3.4. Removing Noisy Features Using Binary Cuckoo Search (BCS) Algorithm. In the term of optimization, FS is defined as maximization task with respect to accuracy (Acc) or a minimization task with respect to the error rate $(1 - \text{Acc})$. Maximization problem can be defined as follows:

$$\text{Maximize } (f(\vec{X})), \tag{8}$$

where $f(\vec{X})$ is the fitness function of $\vec{X}$ that is used to evaluate the solution $\vec{X}$ and the designing of f varies according to nature given problem. In evolutionary computation, the solution is posed of d -decision variables $(x_1, x_2, x_3, \dots, x_d)$ in the search space and x_i is initialized as follows:

$$x_i = \min + (\max - \min) * r, \tag{9}$$

where min and max are lower and upper bounds of x_i and r is a random number that is enclosed in $(0, 1)$. Moreover, optimization algorithms have proved their capabilities in producing reasonable solutions for various optimization problems in a reasonable amount of time.

Population-based algorithms start by generating the initial set of solutions based on randomness (random guess), consequently operating a sequence of procedures in order to produce better generation. In this work, each solution is a vector of binary bits (0 or 1) with a length of d (total number of features). Each bit value belongs to a single attribute. And the initial population is represented as follows:

$$\begin{array}{ccccccc} \vec{X}_2 & 0 & 1 & 0 & 1 & \dots, & 1, \\ \vec{X}_1 & 1 & 1 & 0 & 0 & \dots, & 1. \end{array} \quad (10)$$

Precisely, the value of x_i is determined whether this feature is selected ($x_i = 1$) in the solution $\vec{X}$ or not.

3.4.1. Fitness Function. In FS, the optimal set of features in the subset maximizes the Acc or minimizes the error rate and dips the volume of the given dataset. f is defined as follows:

$$f(\vec{X}) = \max \left(\text{Acc}(\vec{X}) + wf * \left(1 - \frac{l_f}{l_t} \right) \right), \quad (11)$$

where $wf \in (0, 1)$ is the weight factor, l_f is the length of the current subset, and $l_t = d$ is the length of the given dataset. For example, assume a dataset of 40 attributes; $wf = 1$; two solutions provided 97% and 97% accuracies, respectively, where the first one has 20 features, and the second one has 30 features; then f of the first one is 97.5% and f of the second one is 97.25%, which means that first solution is better than the second one. In this study, we utilized the KNN algorithm with 10-fold cross validation.

KNN is one of the easiest ML algorithms to handle yet tends to perform favorably (relatively) [53]. KNN can be used to solve different tasks such as classification, regression, and searching problems. It is considered as a lazy algorithm [54, 55]. Therefore, nothing occurs, or no calculations are conducted in the training phase till the moment we want to predict something (testing phase), which means that no training phase is involved with KNN algorithm. KNN starts by calculating distances when it reaches the first point in the testing phase to determine which class that the current data point belongs to via calculating the similarity (distance) between the data point and all other points that are booked for training portion; then based on a predefined K value, KNN decides which class that the point refers to. KNN used various distance functions such as Euclidean distance (ED), Mahalanobis distance, Manhattan distance, Minkowski distance, Chebyshev distance, cosine distance, correlation distance, Hamming distance, Jaccard distance, standardized ED, and Spearman distance [56]. We set K to 1 and we used ED measurement that is defined as follows:

$$\text{dis}(\vec{p}, \vec{q}) = \sqrt{\sum_{i=1}^d (q_i - p_i)^2}, \quad (12)$$

where $\text{dis}(\vec{p}, \vec{q})$ is the distance between $\vec{p}, \vec{q}$ data points. $(q_i - p_i)^2$ is the square of the subtraction between the vectors of feature of the data points. Two issues must be taken into consideration when modeling KNN that are bias and variant because training error is increased (bias increased) by increasing K value, but testing error may be decreased (variant decreased) and vice versa [57].

3.4.2. CS Algorithm. CS is a swarm intelligence algorithm that is inspired by the reproduction mechanism of cuckoo bird [46]. Cuckoos are mean birds that lay their eggs in other birds' nests and increase their egg probability of being hatched by dropping other bird eggs if necessary. Once the host bird finds out that the nest includes eggs that belong to another bird, it either throws them (intruder eggs) or abandons the nest to build new one in new place. Cuckoo bird decreases the probability of discovering their eggs by fooling the other birds via mimicking the shape pattern and the color of the host eggs. In addition to that, cuckoo eggs hatch earlier than the host eggs and the chicks start throwing the host eggs out of the nest in order to increase their portion of food which is brought by the host bird. Based on the previous analogy, CS has the following procedures:

- Each cuckoo randomly lays one of its eggs to one nest that is chosen by itself each time for reproduction.
- For a fixed number of nests, the high-quality nests will be carried out to the next generation.
- Once a host bird discovers the intruder egg, it throws it or leaves the nest and builds a new one.

The mathematical modeling of CS starts by initializing the population randomly and then each nest (solution) is modified by the Lévy flight (long random walk) function that is as follows:

$$x_{i,\text{new}} = x_{i,\text{old}} + A * \text{levy}(\lambda), \quad (13)$$

where $A > 0$ is the step size scaling parameter, $\text{levy}(\lambda)$ is the Lévy flight term that improves the global search phase, and $\lambda \in [1, 3]$. Random step length is drawn from the Lévy function that is defined as follows:

$$\text{levy}(\lambda) = \text{iter}^{-\lambda}. \quad (14)$$

Each modified nest is compared with a random one $\vec{X}_v$, and $\vec{X}_v$ is replaced by the modified nest only and only if the fitness of new one is greater than the fitness of $\vec{X}_v$.

Finally, nests with the lowest quality of eggs according to a probability pa are discovered by the host bird that are abandoned and replaced by new ones using the local random walk function that is defined as follows:

$$x_{i,\text{new}} = \begin{cases} x_{i,\text{old}} + \delta * (x_{v1} - x_{v2}), & \text{rand} < pa, \\ \text{do nothing}, & \text{rand} \geq pa, \end{cases} \quad (15)$$

where δ is a uniform distribution number $(0, 1)$, x_{v1} and x_{v2} are two randomly selected solutions, and $\text{rand} \in [0, 1]$.

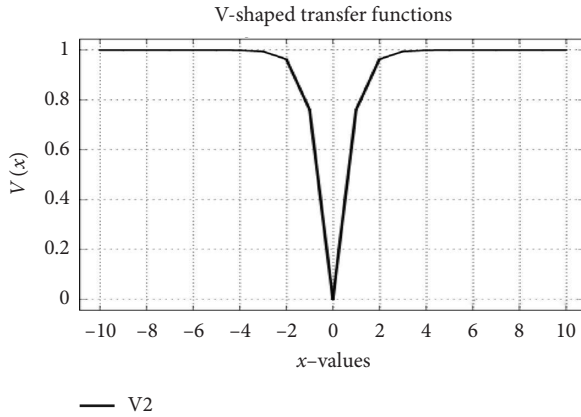
3.5. Discretization via V2 Transfer Function (TF). Naturally, metaheuristic algorithms modify the solutions by a set of procedures that produce evolved generation of continuous values. Hence, these values must be converted to binary ones [58]. TFs are commonly used to encode values to binary without changing the structure of the algorithm. In this work, we used one V-TF called V_2 , $V_2 = |\tan(x_i)|$. See Figure 4.

```

Set the initial values of  $A, \lambda, \delta, pa, Max_{iter}$ , and  $N$  nests.
Evaluate all nests  $\vec{X}$  based on a fitness function  $f$ 
While iter <  $Max_{iter}$  do
  For  $i = 1$  to  $N$  nests do
    Update  $x_{i,new}$  using equation (13) for each solution  $\vec{X}_i$ 
    Select random nest  $\vec{X}_v$ 
    Compare it with  $\vec{X}_i$  based on their  $f$ 
    Update  $\vec{X}_i$  if  $(f(\vec{X}_i) > f(\vec{X}_v))$ 
    Leave worst nests  $\vec{X}_i$  based on  $pa$ 
    Select random solutions  $\vec{X}_{v1}$  and  $\vec{X}_{v2}$ 
    Construct a new nest using equation (15)
  End for
End while
Evaluate each nest and return the optimal one

```

ALGORITHM 1: The pseudocode of CS algorithm.

FIGURE 4: V2 TF. If the result of this function on x_i is greater than 0.5, then $x_i = 1$; else $x_i = 0$.

3.6. FM Classifier. FM is a ML algorithm that can be leveraged for different tasks such as classification, regression, and others. The algorithm was proposed by Hocking [59]; for more information, see https://d2l.ai/chapter_recommender-systems/fm.html. FM is a combination of the linear regression (LR) and the matrix factorization (MF) [60] approaches. Currently, it is utilized by many companies for advertisement and product recommendations since it has several advantages over LR and MF such as that it can model d -way variable interactions, where d is the number of polynomial order and is usually set to 2, and a fast optimization algorithm associated with FMs can reduce the polynomial computation time to linear complexity, making it extremely efficient especially for high-dimensional sparse inputs (huge or massive dataset).

For clear understanding of FM, let $z \in R^d$ denote the feature vectors of one instance and y denote the corresponding target label; then a 2-degree FM is defined as follows:

$$\hat{y}(z) = w_0 + \sum_{i=1}^d w_i z_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle v_i, v_j \rangle z_i z_j, \quad (16)$$

where $w_0 \in R$ is the global bias, $w \in R^d$ depicts the weight of the i^{th} variable, $V \in R^{d \times k}$ depicts the feature embeddings, v_i depicts the i^{th} row of V , k is the dimensionality of latent factors, $\langle \cdot, \cdot \rangle$ is the dot product of two vectors, and $\langle v_i, v_j \rangle$ depicts the intersection between i^{th} and j^{th} features.

Intersection among features is time consuming especially when the features are hidden. Consequently, designing an automatic feature intersection approach decreases the efforts for feature engineering. Computing and identifying all pairwise intersections among features lead to a complexity of $O(Kd^2)$. And reorganizing the third term of equation (16) may decrease the complexity to a linear time $O(kd)$. The redefinition of the third part of FM in equation (16) is as follows:

$$\begin{aligned}
\sum_{j=i+1}^d \langle v_i, v_j \rangle z_i z_j &= \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \langle v_i, v_j \rangle z_i z_j - \frac{1}{2} \sum_{i=1}^d \langle v_i, v_i \rangle z_i z_i \\
&= \frac{1}{2} \left(\sum_{i=1}^d \sum_{j=1}^d \sum_{t=1}^k v_{i,t} v_{j,t} z_i z_j - \sum_{i=1}^d \sum_{t=1}^k v_{i,t} v_{i,t} z_i z_i \right) \\
&= \frac{1}{2} \sum_{t=1}^k \left(\left(\sum_{i=1}^d v_{i,t} z_i \right) \left(\sum_{j=1}^d v_{j,t} z_j \right) - \sum_{i=1}^d v_{i,t}^2 z_i^2 \right) \\
&= \frac{1}{2} \sum_{t=1}^k \left(\left(\sum_{i=1}^d v_{i,t} z_i \right)^2 - \sum_{i=1}^d v_{i,t}^2 z_i^2 \right). \quad (17)
\end{aligned}$$

Upon equation (9), FM complexity is significantly dipped. And for scattered features, only nonzero items require computation which means that the total complexity is linear to the number of nonzero features.

3.7. EL. EL is a common approach for improving the performance of ML algorithms; the claim behind it was drawn from the notion of combining multiple ML models to get a better and more efficient one. In other contexts, utilizing individual branch of classifier may provide poor performance; therefore, to tackle this complication, creating a number of individual learners that are known as weak learners because of either they are highly biased or have high variance [61], yet EL can decrease the biasness and the variance if the model has high bias and low variance and vice versa. Consequently, the resulted learner will be balanced which has low bias and variance. Moreover, the resulted learner is also known as a strong learner which is able to perform better predictions. In this work, we utilized bagging (also known as bootstrap aggregation) technique [62] to combine weak versions of FM classifier that may have high variance where bagging is used to provide strong version of FM in which it comes with low variance. For more information, go to <https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/>, and see Figure 5.

The first stage of bagging is bootstrapping, which tends to train each weak learner independently on different subsets (called bootstrap dataset) of features resampled from the original dataset. The second stage of bagging is aggregation, which contains the prediction process in which each learner makes independent predictions. Subsequently, the prediction results are aggregated to get overall prediction among all weak learners using either max voting or averaging [63]. In this work, we used max voting that depends on the model score of the prediction results of all weak learners. Each prediction score is counted as one vote, and the most reoccurring vote is marked as the corresponding learner (the strongest). And the number of weak learners in this study is 5.

4. Experimental Results

In the following sections, we introduce the following topics to explain how we carry out the experiments to develop and test our model:

- The tools that were utilized to carry out all experiments.
- The characteristics of the used datasets of plant leaf images (Flavia [64] and Swedish [65] datasets).
- The comparisons between different approaches.

4.1. Tools for Carrying Out All Experiments. The tools we utilized in our study are (1) Orange Data Mining, (2) JDK 17, (3) IntelliJ IDEA 2021.2.4, and (4) Oracle ML library called Tribuo. Tribuo is a general purpose open-source ML library written in Java by ML Oracle research group that can be used for classification, clustering, regression, DL, and natural language processing [66].

4.2. Datasets. In our experiments, we used two common and widely used benchmark datasets of plant leaf images, Flavia and Swedish leaf datasets which were employed for assessing our model and for comparisons with other methods. Flavia dataset includes 1907 images referring to 32 different types of plants (labels), where the number of samples in each class varies from 50 to 77. Figure 5 depicts some instances from this dataset. Swedish dataset contains 1125 images referring to 15 classes, where the number of samples in each category is 75. Figure 6 demonstrates some images from this dataset. Images in both sets are very similar in texture and shape features which in turn make the process of leaf classification very challenging.

4.3. Evaluation Measurement. In order to assess the proposed model, we used the Acc and the confusion matrix (CM). Acc is

$$\text{Acc} = \frac{\text{CPS}}{\text{TS}}, \text{ where CPS is correctly predicted samples and TS is the total number of samples, } \quad (18)$$

$$\text{Macro average recall} = \frac{1}{Z} \sum_{i=1}^Z \text{recall}(c_i), \text{ recall}(c_i) = \frac{\text{TP}(c_i)}{\text{TP}_i + \text{FN}_i}, \quad (19)$$

$$\text{Macro average precision} = \frac{1}{Z} \sum_{i=1}^Z \text{precision}(c_i), \text{ precision}(c_i) = \frac{\text{TP}(c_i)}{\text{TP}_i + \text{FP}_i}, \quad (20)$$

$$\text{Macro average } F_1 = \frac{1}{Z} \sum_{i=1}^Z F_1(c_i), F_1(c_i) = 2 \frac{\text{precision}(c_i) * \text{recall}(c_i)}{\text{precision}(c_i) + \text{recall}(c_i)}, \quad (21)$$

where c_i is the i th class. To assess the performance of our FS + IBCS framework, we applied various metrics, including accuracy, F1 score, precision, and recall. These metrics are

derived from the CM, an essential tool for evaluating ML models. The CM consists of four key elements: true positive (TP), true negative (TN), false positive (FP), and false

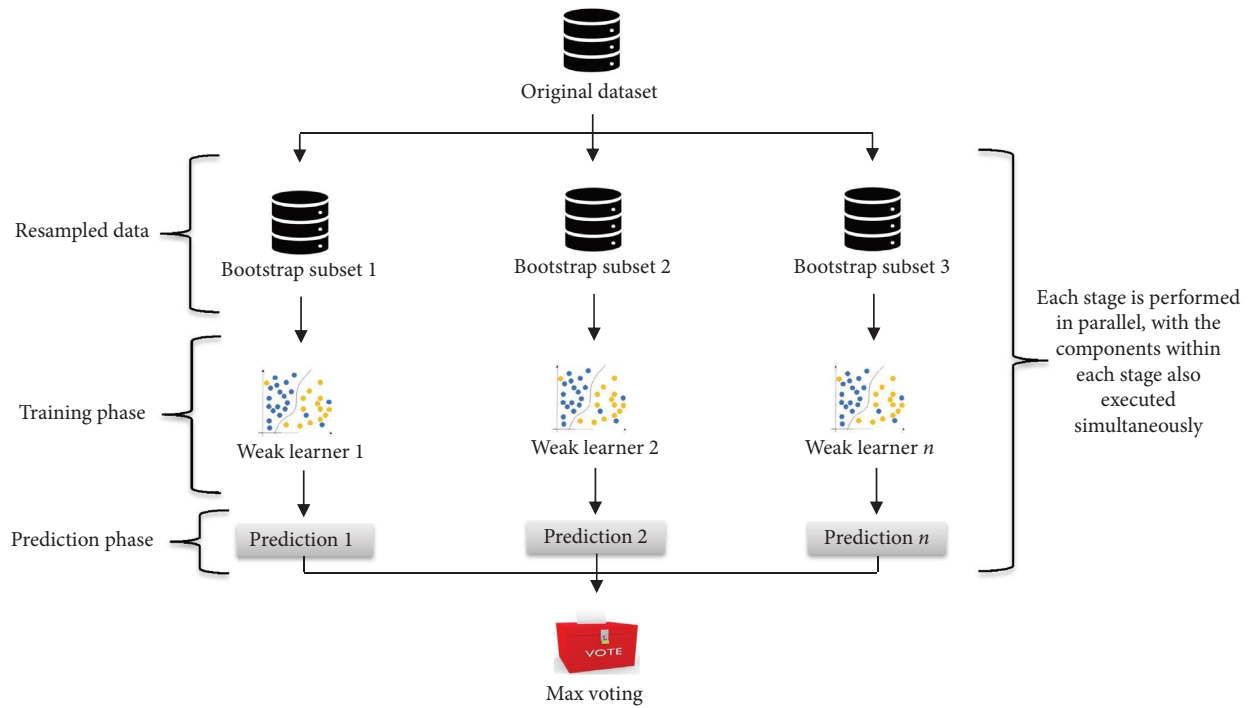


FIGURE 5: The demonstration of EL-based bagging and max voting.

negative (FN). For instance, in a medical diagnosis scenario, TP represents correctly identified infected individuals, TN denotes correctly recognized healthy individuals, FP refers to healthy individuals wrongly diagnosed as infected, and FN indicates infected individuals incorrectly classified as healthy.

CM is a two-dimensional array that informs where the ML model gets confused and demonstrates the distribution of the predictions of the classifier, and it maps the predictions to the categories in which the predicted samples belong [67]. CM can be visualized for clear understanding as shown in Figure 6.

To perform a fair comparison with the state-of-the-art approaches on the Swedish leaf dataset, the datasets were divided into train-test portions in which for each class, 25 instances were randomly selected, and the remaining 50 instances were selected for testing. mRMR was configured to select the most 500 dominance features. $N = 30$, $\text{Max}_{\text{iter}} = 20$, $A = 2$, $\lambda = 2$, $\delta = 1.5$, and $\text{pa} = 0.1$ were used as the parameter settings for BCS algorithm. Hinge loss function [68], Adagrad [69] with initial learning rate equal to 0.1 and epsilon equal to 0.5, number of epochs equal to 50, factorization dimension size equal to 10, and variance equal to 0.2 were used as the parameter settings of the FM classifier. The experiment was repeated 10 times and the mean accuracy was used for the comparisons with some other approaches which had been discussed in Section 2.

According to Table 1, our model achieved the best classification accuracy with 95.67% and outperforms the other competing approaches. The accuracy of our model is 0.24% higher than that in [10]; 0.43% higher than that in [23]; 3.8% higher than that in [24]; and 18.43% significantly

	C1	C2	C3
C1	25	1	2
C2	4	30	2
C3	0	0	17

Prediction

FIGURE 6: CM, where the diagonal poses the correct predictions of ML algorithm for some problems.

higher than that in [22]. Yet our model used a greater number of features.

To perform a fair comparison on Flavia leaf dataset, the data were divided into two parts where the number of samples that were randomly selected for training is 40 instances from each class and 10 samples from the remaining ones were randomly selected from each category for testing.

According to Table 2, the proposed model outperformed the state-of-the-art approaches and achieved the highest accuracy in comparison to other approaches with 99.6 accuracy which is an indicator that our model is significantly better than those compared. Our model achieved an accuracy of 99.6%, which represents a significant improvement over existing methods. Specifically, it outperformed Xu [5] by 5.57%, Liu et al. [74] by 11.68%, Lavania and Matey [73] by 12.1%, Wang [72] by 13.97%, Kheirkhah and Asghari [71] by 14.9%, and Pham et al. [70] by 14.91%. Furthermore, according to previous comparisons, our proposed model proves that it can be utilized as an efficient alternative to the

TABLE 1: Comparison-based accuracy on Swedish leaf dataset.

Approach	Number of features	AVG Acc %	AVG recall %	AVG precision %	AVG F_1 %
Rojas-Hernandez et al. [17]	8	77.24	N/A	N/A	N/A
Shao [18]	80	95.24	N/A	N/A	N/A
Xu [5]	35	95.43	N/A	N/A	N/A
Ours (FS + mRMR)	500	95.50	95.48	95.49	95.48
Ours (FS + IBCS)	1051	95.67	95.66	95.67	95.67

TABLE 2: Comparison-based accuracy on Flavia leaf dataset.

Approach	Number of features	Acc %	AVG recall %	AVG precision %	AVG F_1 %
Pham et al. [70]	Unknown	84.69	N/A	N/A	N/A
Mostajer Kheirkhah and Asghari [71]	Unknown	84.70	N/A	N/A	N/A
Wang et al. [72]	Unknown	85.63	N/A	N/A	N/A
Lavana and Matey [73]	Unknown	87.50	N/A	N/A	N/A
Liu et al. [74]	Unknown	87.92	N/A	N/A	N/A
Xu [5]	Unknown	94.03	N/A	N/A	N/A
Ours (FS + mRMR)	500	98.20	98.19	98.20	98.18
Ours (FS + IBCS)	1024	99.6	99.65	99.67	99.66

existing state-of-the-art approaches which had been proposed for tackling the plant leaf classification problem in the existing literature.

5. Limitation and Discussion

5.1. Rationale of Adopting CS Over Other Methods. CS is a relatively new optimization algorithm that demonstrates efficient performance in solving complex tasks, such as the 0/1 knapsack problem [75] and the traveling salesman problem [76]. Unlike the particle swarm optimization algorithm, which often converges to near-optimal solutions due to its evolving nature and inability to fully satisfy global convergence conditions [77], CS offers robust search capabilities that frequently lead to global convergence [78]. This helps it overcome premature convergence and the issue of local minima. Additionally, CS is simple to implement [79], as its evolving procedure involves only two equations: Lévy flights and random walks. These two search mechanisms explore the search space more effectively than calculus-based techniques, which rely on gradient calculations that are sensitive to starting points. By utilizing randomness-based strategies, CS avoids local minima and navigates large search spaces efficiently, increasing the likelihood of finding near-global optimal solutions [80]. Indeed, many works have reported the efficiency of CS for solving diverse complex problems [81–84].

5.2. Adopting CS for Unsupervised Learning. This section discusses implementing our framework in an unsupervised context, where target labels are unknown [85]. The goal is to identify the smallest subset of features that enhances clustering performance by revealing meaningful natural groupings [86]. The process involves (1) inputting the dataset, (2) using a heuristic search to generate feature subsets, and (3) applying a clustering algorithm to each subset based on specific

evaluation criteria. This iterative process identifies the most effective feature subset. The key difference between FS in supervised and unsupervised learning lies in the evaluation methods used. In supervised learning, FS involves classification algorithms like XGBoost, whereas in unsupervised learning, clustering algorithms, such as k-means [87], evaluate the feature subset's effectiveness.

Designing a fitness function in heuristic searches is crucial, as it guides decision making. Various fitness functions have been proposed for this task. For instance, in [85], an expectation-maximization clustering algorithm assesses feature subsets, assuming the optimal set follows a Gaussian distribution and maximizes likelihood. In [88], a CS algorithm maintains different feature subsets, using a fitness function that minimizes the intra_{cluster} + partition_{index} and maximizes the partition_{coefficient}. Additionally, we recommend minimizing both the number of features in the subset and the correlation between them.

5.3. Local Optima and Premature Convergence. The BCS algorithm, being a population-based approach, often faces the challenge of getting trapped in local optima, where all solutions converge to the same point, thereby limiting further exploration. In other words, the solutions converge to a single point, restricting the exploration phase and reducing the algorithm's ability to discover broader regions of the search space that are more likely to contain the optimal solution [89, 90]. This lack of diversity in the current solutions at the early stages is referred to as premature convergence. This phenomenon can be understood as follows: The approach begins by generating initial solutions without prior knowledge of better solutions. Due to the nature of the algorithm, these solutions quickly converge to the same single solution, leading to a focus on limited regions of the search space and an increased likelihood of getting trapped in local optima [91, 92].

TABLE 3: The performance of IBCSA with various K -fold validation on Swedish leaf dataset.

K-fold validation	AVG Acc %	AVG recall %	AVG precision %	AVG F_1 %
$K = 3$	98.80	96.73	96.73	96.93
$K = 5$	97.50	97.50	97.50	97.50
$K = 7$	97.24	97.41	97.22	97.40
$K = 9$	97.42	97.60	97.50	97.80

TABLE 4: The performance of IBCSA with various K -fold validation on Flavia leaf dataset.

K-fold validation	AVG Acc %	AVG recall %	AVG precision %	AVG F_1 %
$K = 3$	96.07	92.92	92.94	93.44
$K = 5$	96.70	93.50	93.44	94.02
$K = 7$	96.70	93.91	93.70	94.05
$K = 9$	97.60	94.40	94.30	94.73

5.4. Drawbacks of the Fitness Function and Correlation Awareness. One limitation of our approach lies in the design of the fitness function. While it is a multiobjective function that aims to maximize accuracy $\text{Acc}(X)$ and minimize the number of selected features $wf * (1 - l_f/l_t)$, it does not account for the correlation between features or the correlation between each feature and the corresponding class label [93]. This omission limits the ability of the fitness function to capture the relationships among variables in the selected subset, potentially impacting the quality of the selected features. Additionally, the randomness-based nature of the search algorithm lacks prior knowledge about feature correlations, which can lead to suboptimal decision making during the selection process. Enhancing the fitness function to consider these correlations could address this limitation and improve the overall performance of the approach.

5.5. Using Cross-Validation to Mitigate Overfitting and Assess the Generalizability of the Model. We observed that the presence of outliers in the fitness scores significantly impacted the model's performance, often leading to overfitting. Outliers, which fall outside the expected range of values, tend to distort the model's understanding of the underlying data distribution. In the case of optimization algorithms, this results in the model fitting excessively to the outliers, causing it to focus too heavily on specific noise in the data rather than the general pattern. As a result, the model becomes overly complex and tailored to the training data, losing its ability to generalize effectively to new, unseen data. This overfitting problem not only affects the model's predictive accuracy but also hinders its robustness when deployed in real-world scenarios. Classifiers like KNN are particularly vulnerable to this issue [94], as the presence of outliers leads to biased predictions, making the model less reliable. To mitigate this, it is crucial to ensure that the model maintains simplicity and avoids fitting noise, ensuring it generalizes well to unseen instances and performs consistently over unseen data [95].

Cross-validation is a method used to evaluate a model's ability to generalize and prevent overfitting. It works by dividing the dataset into several subsets or folds, with some folds used for training the model and others for testing. This process is repeated, with each fold being used as the test set

once, and the results are averaged to provide a more reliable measure of the model's performance. By exposing the model to different subsets of data, cross-validation helps it learn broader patterns and avoids fitting too closely to any single subset. It also ensures configurations that enhance generalization and minimizes bias by evaluating the model across varied data subsets, ensuring it performs well on unseen data [96–98]. Here, we used different values for K -fold validation.

Table 3 demonstrates that even with different numbers of cross-validation folds, the IBCSA model shows stable and consistent performance in terms of accuracy, recall, precision, and F_1 score. This indicates that the model becomes better at generalizing to unseen data, as cross-validation with various K -values reduces the risk of overfitting by exposing the model to a broader range of data subsets. To clarify, the discrepancy in accuracy measurements is small (1.56), while in recall, precision, and F_1 score, the discrepancies are 0.87, 0.77, and 0.87, respectively. The consistent performance with low discrepancies across all metrics suggests that the model is able to generalize effectively with more diverse training and testing data. Overall, the IBCSA model exhibits strong generalization and robustness, even with varying K -fold values.

Table 4 illustrates that the IBCSA model maintains stable and consistent performance across accuracy, recall, precision, and F_1 score, regardless of the number of cross-validation folds. This suggests that the model improves its ability to generalize to unseen data, as using different K -values in cross-validation helps mitigate overfitting by exposing the model to a wider variety of data subsets. Specifically, the accuracy discrepancy is minimal (1.53), while recall, precision, and F_1 score show discrepancies of 1.48, 1.36, and 1.29, respectively. The low discrepancies across all metrics highlight the model's effective generalization with more varied training and testing data. Overall, the IBCSA model demonstrates strong robustness and generalization, even when using different K -fold values.

6. Conclusion

Plants and leaves play an indispensable role in the life cycle from producing oxygen to its importance in the medical

industry. With the passage of time, classifying plants by modern techniques in computer vision, image processing, and pattern recognition has attracted the attention of many researchers and encouraged them to work to tackle this task. In this study, we introduced a model that included mRMR filter, BCS algorithm for dropping disturbance features, bagging-based max voting to construct 10 weak learners of FM classifier and then retrieve the best one of them, and Inception V3 network used for FE. The model had been tested on two benchmark datasets, Swedish and Flavia leaf data. The outcomes of this study and the experimental results affirmed that this work could be employed for identifying the plant leaves efficiently. And the proposed model provided higher classification accuracy against other state-of-the-art approaches, as it achieved 99.6 and 95.67 on the datasets, respectively.

For future work, we plan to incorporate the correlations among the features in the selected subset as well as between each feature and the corresponding label to measure the dependency among these variables. In addition, Kendall's, Spearman's, and Pearson's correlation functions can be employed for this purpose, while null space and full-rank matrix approaches can be utilized to uncover the linear dependency among these variables. Additionally, we intend to adopt various techniques to address the challenges of local optima and premature convergence by leveraging genetic-based methods such as crossover, mutation, and opposition-based learning.

Data Availability Statement

The framework implementation and datasets generated during the current study are available in the GitHub repository at <https://github.com/Mohammed-Riyad-Eiadeh/Binary-Cuckoo-Search-For-Plant-Leaf-Prediction>.

Conflicts of Interest

The authors declare no conflicts of interest.

Funding

This work was supported by Jadara University and Prince Mohammad bin Fahad University.

References

- [1] Ş. Işık and K. Özkan, "Overview of Handcrafted Features and Deep Learning Models for Leaf Recognition," *Journal of Engineering Research* 9, no. 1 (2021): 105–116, <https://doi.org/10.36909/jer.v9i1.7737>.
- [2] M. A. F. Azlah, L. S. Chua, F. R. Rahmad, F. I. Abdullah, and S. R. W. Alwi, "Review on Techniques for Plant Leaf Classification and Recognition," *Computers* 8, no. 4 (2019): <https://doi.org/10.3390/computers8040077>.
- [3] M. S. Al-Batah, M. Alzyoud, R. Alazaidah, M. Toubat, H. Alzoubi, and A. Olaiyat, "Early Prediction of Cervical Cancer Using Machine Learning Techniques," *The Jordanian Journal of Computers and Information Technology* 8, no. 4 (2022): 357–369, <https://doi.org/10.5455/jcit.71-1661691447>.
- [4] F. Sadeghi, A. Larijani, O. Rostami, D. Martín, and P. Hajirahimi, "A Novel Multi-Objective Binary Chimp Optimization Algorithm for Optimal Feature Selection: Application of Deep-Learning-Based Approaches for SAR Image Classification," *Sensors* 23, no. 3 (2023): 1180, <https://doi.org/10.3390/s23031180>.
- [5] G. Xu, "Effective Scale-Generation Scheme Based Shape Descriptor for Fast" (2022), 1–10, <https://doi.org/10.1049/tje2.12241>.
- [6] J. Su, M. Wang, Z. Wu, and Q. Chen, "Fast Plant Leaf Recognition Using Improved Multiscale Triangle Representation and KNN for Optimization," *IEEE Access* 8 (2020): 208753–208766, <https://doi.org/10.1109/ACCESS.2020.3037649>.
- [7] K. Pankaja and V. Suma, "Plant Leaf Recognition and Classification Based on the Whale Optimization Algorithm (WOA) and Random Forest (RF)," *Journal of The Institution of Engineers (India): Series B* 101, no. 5 (2020): 597–607, <https://doi.org/10.1007/s40031-020-00470-9>.
- [8] K. Lin, Y. Zhao, L. Wang, W. Shi, F. Cui, and T. Zhou, "MSWNet: A Visual Deep Machine Learning Method Adopting Transfer Learning Based Upon ResNet 50 for Municipal Solid Waste Sorting," *Frontiers of Environmental Science and Engineering* 17, no. 6 (2023): 1–12, <https://doi.org/10.1007/s11783-023-1677-1>.
- [9] A. Rai, S. Dey, P. Patidar, and P. Rai, "MoSFPAD: An End-to-End Ensemble of MobileNet and Support Vector Classifier for Fingerprint Presentation Attack Detection," 20, no. 10 (2023), 1–12, <https://arxiv.org/abs/2303.01465>.
- [10] S. Mirjalili and A. Lewis, "The Whale Optimization Algorithm," *Advances in Engineering Software* 95 (2016): 51–67, <https://doi.org/10.1016/j.advengsoft.2016.01.008>.
- [11] P. P. Kaur, S. Singh, I. S. Sandhu, and D. S. Sandhanwalia, "Leaf Recognition System Based on Supervised Machine Learning Techniques," 2022 *IEEE Delhi Section Conference (DELCON)* (2022), <https://doi.org/10.1109/DELCON54057.2022.9753062>.
- [12] Z. Ibrahim, N. Sabri, and N. N. A. Mangshor, "Leaf Recognition Using Texture Features for Herbal Plant Identification," *Indonesian Journal of Electrical Engineering and Computer Science* 9, no. 1 (2018): 152–156, <https://doi.org/10.11591/ijeecs.v9.i1.pp152-156>.
- [13] M. Asim, S. Ullah, A. Razzaq, and S. Qadri, "Varietal Discrimination of Guava (Psidium Guajava) Leaves Using Multi Features Analysis," *International Journal of Food Properties* 26, no. 1 (2023): 179–196, <https://doi.org/10.1080/10942912.2022.2158863>.
- [14] C. Yang, "Plant Leaf Recognition by Integrating Shape and Texture Features," *Pattern Recognition* 112 (2021): 107809, <https://doi.org/10.1016/j.patcog.2020.107809>.
- [15] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *International Conference on Machine Learning (ICML 2015)*, 1 (2015), 448–456.
- [16] X. Zhu, M. Zhu, and H. Ren, "Method of Plant Leaf Recognition Based on Improved Deep Convolutional Neural Network," *Cognitive Systems Research* 52 (2018): 223–233, <https://doi.org/10.1016/j.cogsys.2018.06.008>.
- [17] R. Rojas-Hernandez, A. Lopez-Chau, V. Trujillo-Mora, and C. A. Rojas-Hernandez, "Plant Identification Using New Geometric Features With Standard Data Mining Methods," *IEEE International Conference on Networking, Sensing and Control (ICNSC 2016)* (2016): 1–4, <https://doi.org/10.1109/ICNSC.2016.7479024>.
- [18] Y. Shao, "Supervised Global-Locality Preserving Projection for Plant Leaf Recognition," *Computers and Electronics in*

- Agriculture* 158 (2018): 102–108, <https://doi.org/10.1016/j.compag.2019.01.022>.
- [19] J. Cao, B. Wang, and D. Brown, “Similarity Based Leaf Image Retrieval Using Multiscale R-Angle Description,” *The Information of the Science* 374 (2016): 51–64, <https://doi.org/10.1016/j.ins.2016.09.023>.
 - [20] X. Wang, B. Feng, X. Bai, W. Liu, and L. Jan Latecki, “Bag of Contour Fragments for Robust Shape Classification,” *Pattern Recognition* 47, no. 6 (2014): 2116–2125, <https://doi.org/10.1016/j.patcog.2013.12.008>.
 - [21] N. Alajlan, I. El Rube, M. S. Kamel, and G. Freeman, “Shape Retrieval Using Triangle-Area Representation and Dynamic Space Warping,” *Pattern Recognition* 40, no. 7 (2007): 1911–1920, <https://doi.org/10.1016/j.patcog.2006.12.005>.
 - [22] R. Poli, J. Kennedy, and T. Blackwell, “Quantification and Assessment of the Chemical Form of Residual Gadolinium in the Brain.pdf,” *Swarm Intelligence* 1 (2007): 33–57, <https://doi.org/10.1007/s11721-007-0002-0>.
 - [23] Y. Shi, E. M. Izhikevich, G. B. Fogel, and J. M. Zurada, “The Newsletter of the IEEE Neural Networks Society A New Era: Intelligence President’s Message,” *Neural Networks* 2, no. 1 (2004): 1–2.
 - [24] C. C. O. Ramos, D. Rodrigues, A. N. De Souza, and J. P. Papa, “On the Study of Commercial Losses in Brazil: A Binary Black Hole Algorithm for Theft Characterization,” *IEEE Transactions on Smart Grid* 9, no. 2 (2018): 676–683, <https://doi.org/10.1109/TSG.2016.2560801>.
 - [25] E. D. Goodman, “Introduction to Genetic Algorithms,” *Genetic Improvement of Software: A Comprehensive Survey* (2007), 3205–3224, <https://doi.org/10.1145/1274000.1274111>.
 - [26] Y. T. Hsiao, C. L. Chuang, J. A. Jiang, and C. C. Chien, “A Novel Optimization Algorithm: Space Gravitational Optimization,” *IEEE International Conference on Systems, Man and Cybernetics, IEEE Conference*, 3 (IEEE Xplore, 2014), 2323–2328, <https://doi.org/10.1109/icsmc.2005.1571495>.
 - [27] S. Selvarajan, *A Comprehensive Study on Modern Optimization Techniques for Engineering Applications* (Netherlands: Springer, 2024).
 - [28] J. Ramasamy, R. N. Ravikumar, and S. Shitharth, “Artificial Neural Networks for Data Processing: A Case Study of Image Classification,” in *Advanced Mathematical Applications in Data Science* (Bentham Science Publishers, 2023), 116–127.
 - [29] S. Shitharth, H. Manoharan, A. M. Alshareef, A. Yafaz, H. Alkhiri, and O. M. Mirza, “Hyper Spectral Image Classifications for Monitoring Harvests in Agriculture Using Fly Optimization Algorithm,” *Computers and Electrical Engineering* 103 (2022): 108400.
 - [30] C. Szegedy, V. Vanhoucke, S. Ioffe, and J. Shlens, “InceptionV2/BN-Inception” (2015).
 - [31] S. Karen and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition Karen,” *American Journal of Health-System Pharmacy* 75, no. 6 (2018): 398–406.
 - [32] M. S. Brandon Amos and B. Ludwiczuk, “Openface: Uniwersalna Biblioteka Rozpoznawania Twarzy Z Aplikacjami Mobilnymi,” *C Tutorial* 16 (2015): 1–18, <https://reports-archive.adm.cs.cmu.edu/anon/anon/usr0/ftp/2016/CMU-CS-16-118.pdf>.
 - [33] S. García, J. Luengo, and F. Herrera, “Data Preprocessing in Data Mining” (2015).
 - [34] M. B. Kursu and W. R. Rudnicki, “Feature Selection With the Boruta Package,” *Journal of Statistical Software* 36, no. 11 (2010): 1–13, <https://doi.org/10.18637/jss.v036.i11>.
 - [35] Y. Gao, Y. Zhou, and Q. Luo, “An Efficient Binary Equilibrium Optimizer Algorithm for Feature Selection,” *IEEE Access* 8 (2020): 140936–140963, <https://doi.org/10.1109/ACCESS.2020.3013617>.
 - [36] K. Lillywhite, D.-J. Lee, B. Tippetts, and J. Archibald, “A Feature Construction Method for General Object Recognition,” *Pattern Recognition* 46, no. 12 (2013): 3300–3314, <https://doi.org/10.1016/j.patcog.2013.06.002>.
 - [37] H. Motoda and H. Liu, “Feature Selection, Extraction and Construction,” *Communism* 5 (2002): 67–72.
 - [38] S. Khalid, T. Khalil, and S. Nasreen, “A Survey of Feature Selection and Feature Extraction Techniques in Machine Learning,” *Proceedings of 2014 Science and Information Conference, SAI 2014* (2014), 372–378, <https://doi.org/10.1109/SAI.2014.6918213>.
 - [39] A. Jović, K. Brkić, and N. Bogunović, “A Review of Feature Selection Methods With Applications,” *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)* (2015), 1200–1205, <https://doi.org/10.1109/MIPRO.2015.7160458>.
 - [40] V. Kumar, “Feature Selection: A Literature Review,” *Smart Computer in Rewa* 4, no. 3 (2014): <https://doi.org/10.6029/smartcr.2014.03.007>.
 - [41] A. Bosin, N. Dessì, and B. Pes, “Intelligent Data Engineering and Automated Learning: IDEAL 2007,” *Ideal* 4881 (2007): 790–799, <https://doi.org/10.1007/978-3-540-77226-2>.
 - [42] S. Maldonado and R. Weber, “A Wrapper Method for Feature Selection Using Support Vector Machines,” *The Information of the Science* 179, no. 13 (2009): 2208–2217, <https://doi.org/10.1016/j.ins.2009.02.014>.
 - [43] H. Liu, M. Zhou, and Q. Liu, “An Embedded Feature Selection Method for Imbalanced Data Classification,” *IEEE/CAA Journal of Automatica Sinica* 6, no. 3 (2019): 703–715, <https://doi.org/10.1109/JAS.2019.1911447>.
 - [44] M. A. Sulaiman and J. Labadin, “Feature Selection Based on Mutual Information,” no. 8 (2015), 1–6, <https://doi.org/10.1109/cita.2015.7349827>.
 - [45] Y. Cai, T. Huang, L. Hu, X. Shi, L. Xie, and Y. Li, “Prediction of Lysine Ubiquitination With mRMR Feature Selection and Analysis,” *Amino Acids* 42, no. 4 (2012): 1387–1395, <https://doi.org/10.1007/s00726-011-0835-0>.
 - [46] X. S. Yang and S. Deb, “Cuckoo Search via Lévy Flights,” *Proceedings of World Congress on Nature & Biologically Inspired Computing (NaBIC 2009, India)* (2009), 210–214, <https://doi.org/10.1109/NABIC.2009.5393690>.
 - [47] Y. S. Jeong, K. S. Shin, and M. K. Jeong, “An Evolutionary Algorithm With the Partial Sequential Forward Floating Search Mutation for Large-Scale Feature Selection Problems,” *Journal of the Operational Research Society* 66, no. 4 (2015): 529–538, <https://doi.org/10.1057/jors.2013.72>.
 - [48] P. Bugata and P. Drotar, “On Some Aspects of Minimum Redundancy Maximum Relevance Feature Selection,” *Science China Information Sciences* 63, no. 1 (2020): 1–15, <https://doi.org/10.1007/s11432-019-2633-y>.
 - [49] D. Reduction, “Correlation Based Feature,” 32–33.
 - [50] J. R. Vergara and P. A. Estévez, “A Review of Feature Selection Methods Based on Mutual Information,” *Neural Computing and Applications* 24, no. 1 (2014): 175–186, <https://doi.org/10.1007/s00521-013-1368-0>.
 - [51] J. Jaeger, R. Sengupta, and W. L. Ruzzo, “Impro Ved G Ene Selectio N Fo R Classificatio N O F Microarrays”.
 - [52] M. Radovic, M. Ghalwash, N. Filipovic, and Z. Obradovic, “Minimum Redundancy Maximum Relevance Feature

- Selection Approach for Temporal Gene Expression Data," *BMC Bioinformatics* 18, no. 1 (2017): 1–14, <https://doi.org/10.1186/s12859-016-1423-9>.
- [53] Z. Deng, X. Zhu, D. Cheng, M. Zong, and S. Zhang, "Efficient kNN Classification Algorithm for Big Data," *Neurocomputing* 195 (2019): 143–148, <https://doi.org/10.1016/j.neucom.2015.08.112>.
 - [54] M. L. Zhang and Z. H. Zhou, "ML-KNN: A Lazy Learning Approach to Multi-Label Learning," *Pattern Recognition* 40, no. 7 (2007): 2038–2048, <https://doi.org/10.1016/j.patcog.2006.12.019>.
 - [55] L. Xiong, S. Chitti, and L. Liu, "Mining Multiple Private Databases Using a kNN Classifier," *Proceedings of the ACM Symposium on Applied Computing* (2007): 435–440, <https://doi.org/10.1145/1244002.1244102>.
 - [56] H. A. Abu Alfeilat, et al., "Effects of Distance Measure Choice on K-Nearest Neighbor Classifier Performance: A Review," *Big Data* 7, no. 4 (2019): 221–248, <https://doi.org/10.1089/big.2018.0175>.
 - [57] "Day-3-k-Nearest-Neighbors-and-Bias-Variance-Tradeoff-75f84d515bdb @ medium.Com".
 - [58] J. Jafari-Asl, G. Azizyan, S. A. H. Monfared, M. Rashki, and A. G. Andrade-Campos, "An Enhanced Binary Dragonfly Algorithm Based on a V-Shaped Transfer Function for Optimization of Pump Scheduling Program in Water Supply Systems (Case Study of Iran)," *Engineering Failure Analysis* 123 (2021): 105323, <https://doi.org/10.1016/j.engfailanal.2021.105323>.
 - [59] R. R. Hocking, "A Biometrics Invited Paper. The Analysis and Selection of Variables in Linear Regression," *Biometrics* 32, no. 1 (1976): 1, <https://doi.org/10.2307/2529336>.
 - [60] D. Lee and H. Seung, "Learning the Pars of Objects by Non-negative Matrix Factorization," *Nature* 401 (1999): 788–791.
 - [61] T. G. Dietterich and E. B. Kong, "Machine Learning Bias, Statistical Bias, and Statistical Variance of Decision Tree Algorithms" (1995).
 - [62] V. N. Vapnik, "ESL_Statistics the Elements of Statistical Learning," *The Mathematical Intelligencer* 27, no. 2 (2017): 83–85, <https://www.springerlink.com/index/D7X7KX6772HQ2135.pdf>.
 - [63] R. Polikar, "Ensemble Based Systems in Decision Making," *IEEE Circuits and Systems Magazine* 6, no. 3 (2006): 21–44, <https://doi.org/10.1109/MCAS.2006.1688199>.
 - [64] S. G. Wu, F. S. Bao, E. Y. Xu, Y. X. Wang, Y. F. Chang, and Q. L. Xiang, "A Leaf Recognition Algorithm for Plant Classification Using Probabilistic Neural Network," *2007 IEEE International Symposium on Signal Processing and Information Technology* (2007), 11–16, <https://doi.org/10.1109/ISSPIT.2007.4458016>.
 - [65] O. J. O. Söderkvist, "Computer Vision Classification of Leaves From Swedish Trees" (2001).
 - [66] M. Szabó, "Machine Learning on Android With Oracle Tribuo, SMILE and Weka," *CEUR Workshop Proceedings* 2874 (2021): 176–186.
 - [67] V. M. Patro and M. Ranjan Patra, "Augmenting Weighted Average With Confusion Matrix to Enhance Classification Accuracy," *Transactions on Machine Learning Research* 2, no. 4 (2014): <https://doi.org/10.14738/tmlai.24.328>.
 - [68] R. C. Moore and J. DeNero, "L1 and L2 Regularization for Multiclass Hinge Loss Models" (2009).
 - [69] A. Lydia and S. F. Francis, "Adagrad: An Optimizer for Stochastic Gradient Descent," *International Journal of Computing* 6, no. 5 (2019): 566–568, <https://ijics.com>.
 - [70] N. Pham, T. Le, P. Grard, and V. Nguyen, "Computer Aided Plant Identification System," *International Conference on Computing, Management Telecommunications-Commantel 2013* (Vietnam: IEEE, Hochiminh, Jan 2013).
 - [71] F. Mostajer Kheirkhah and H. Asghari, "Plant Leaf Classification Using GIST Texture Features," *IET Computer Vision* 13, no. 4 (2019): 395–403, <https://doi.org/10.1049/iet-cvi.2018.5028>.
 - [72] B. Wang, D. Brown, Y. Gao, and J. La Salle, "MARCH: Multiscale-Arch-Height Description for Mobile Retrieval of Leaf Images," *The Information of the Science* 302 (2015): 132–148, <https://doi.org/10.1016/j.ins.2014.07.028>.
 - [73] S. Lavania and P. S. Matey, "Leaf Recognition Using Contour Based Edge Detection and SIFT Algorithm," *2014 IEEE International Conference on Computational Intelligence and Computing Research, IEEE ICCIC 2014* (2015), <https://doi.org/10.1109/ICCIC.2014.7238345>.
 - [74] J. Liu, S. Yang, Y. Cheng, and Z. Song, "Plant Leaf Classification Based on Deep Learning," *Proceedings of the 2018 Chinese Automation Congress (CAC, 2018)*, 3165–3169, <https://doi.org/10.1109/CAC.2018.8623427>.
 - [75] A. Gherboudj, A. Layeb, and S. Chikhi, "Solving 0-1 Knapsack Problems by a Discrete Binary Version of Cuckoo Search Algorithm," *International Journal of Bio-Inspired Computation* 4, no. 4 (2012): 229–236, <https://doi.org/10.1504/IJBIC.2012.048063>.
 - [76] A. Ouaraab, B. Ahiod, and X.-S. Yang, "Discrete Cuckoo Search Algorithm for the Travelling Salesman Problem," *Neural Computing and Applications* 24 (2014): 1659–1669.
 - [77] X.-S. Yang and S. Deb, "Cuckoo Search: Recent Advances and Applications," *Neural Computing and Applications* 24 (2014): 169–174.
 - [78] X. S. Yang and S. Deb, "Engineering Optimisation by Cuckoo Search," *International Journal of Mathematical Modelling and Numerical Optimisation* 1, no. 4 (2010): 330–343, <https://doi.org/10.1504/IJMMNO.2010.035430>.
 - [79] M. Shehab, A. T. Khader, and M. A. Al-Betar, "A Survey on Applications and Variants of the Cuckoo Search Algorithm," *Applied Soft Computing* 61 (2017): 1041–1059.
 - [80] F. Ahmad, N. A. M. Isa, M. K. Osman, and Z. Hussain, "Performance Comparison of Gradient Descent and Genetic Algorithm Based Artificial Neural Networks Training," in *2010 10th International Conference on Intelligent Systems Design and Applications* (2010), 604–609, <https://doi.org/10.1109/ISDA.2010.5687199>.
 - [81] A. H. Gandomi, X.-S. Yang, and A. H. Alavi, "Cuckoo Search Algorithm: A Metaheuristic Approach to Solve Structural Optimization Problems," *Engineering Computers* 29 (2013): 17–35.
 - [82] E. Valian, S. Mohanna, and S. Tavakoli, "Improved Cuckoo Search Algorithm for Global Optimization," *International Journal of Communication and Information Technology* 1, no. 1 (2011): 31–44.
 - [83] X.-S. Yang and S. Deb, "Multiobjective Cuckoo Search for Design Optimization," *Computers and Operations Research* 40, no. 6 (2013): 1616–1624.
 - [84] M. Basu and A. Chowdhury, "Cuckoo Search Algorithm for Economic Dispatch," *Energy* 60 (2013): 99–108.
 - [85] H. B. Barlow, "Unsupervised Learning," *Neural Comput* 1, no. 3 (1989): 295–311, <https://doi.org/10.1162/neco.1989.1.3.295>.
 - [86] J. G. Dy and C. E. Brodley, "Feature Selection for Unsupervised Learning," *Journal of Machine Learning Research* 5 (2004): 845–889.
 - [87] S. Alelyani, J. Tang, and H. Liu, "Feature Selection for Clustering: A Review," *Data Cluster* (2018): 29–60.
 - [88] L. Khriisi, N. E. L. Akkad, H. Satori, and K. Satori, "Simple and Efficient Clustering Approach Based on Cuckoo Search Algorithm," in *2020 Fourth International Conference on*

- Intelligent Computing in Data Sciences (ICDS)* (2020), 1–6, <https://doi.org/10.1109/ICDS50568.2020.9268754>.
- [89] M. Rocha and J. Neves, “Preventing Premature Convergence to Local Optima in Genetic Algorithms via Random Offspring Generation,” in *Multiple Approaches to Intelligent Systems: 12th International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems IEA/AIE-99* (1999), 127–136.
 - [90] E. Cuevas and A. Reyna-Orta, “A Cuckoo Search Algorithm for Multimodal Optimization,” *The Scientific World Journal* 2014, no. 1 (2014): 497514.
 - [91] G. Rudolph, “Self-Adaptive Mutations May Lead to Premature Convergence,” *IEEE Transactions on Evolutionary Computation* 5, no. 4 (2001): 410–414.
 - [92] C. Vanaret, J.-B. Gotteland, N. Durand, and J.-M. Alliot, “Preventing Premature Convergence and Proving the Optimality in Evolutionary Algorithms,” in *Artificial Evolution: 11th International Conference, Evolution Artificielle* (2013), 29–40.
 - [93] M. R. Al-Eideh, R. Qaddoura, and M. Abdallah, “Investigating the Performance of a Novel Modified Binary Black Hole Optimization Algorithm for Enhancing Feature Selection,” *Applied Sciences* 14, no. 12 (2024): 5207.
 - [94] G. Brown, A. Pocock, M.-J. Zhao, and M. Luján, “Conditional Likelihood Maximisation: A Unifying Framework for Information Theoretic Feature Selection,” *Journal of Machine Learning Research* 13, no. 1 (2012): 27–66.
 - [95] G. Sahebi, P. Movahedi, M. Ebrahimi, T. Pahikkala, J. Plosila, and H. Tenhunen, “GeFeS: A Generalized Wrapper Feature Selection Approach for Optimizing Classification Performance,” *Computers in Biology and Medicine* 125 (2020): 103974.
 - [96] D. Berrar, “Cross-Validation” (2019).
 - [97] M. W. Browne, “Cross-Validation Methods,” *Journal of Mathematical Psychology* 44, no. 1 (2000): 108–132.
 - [98] M. Stone, “Cross-Validation: A Review,” *Statistics: A Journal of Theoretical and Applied Statistics* 9, no. 1 (1978): 127–139.